

Guía de Trabajo con Git, GitHub y Entornos Virtuales

Smartphone Use Prediction UTEC | Estándares de Flujo de Trabajo y Colaboración

1. Configuración Inicial (Solo el primer día)

Para descargar el proyecto en tu computadora por primera vez, abre **Git Bash** en la carpeta donde deseas guardar el repositorio y ejecuta los siguientes comandos:

Bash

```
# 1. Clonar el repositorio remoto en tu máquina local
git clone https://github.com/frankrubio/Smarthphone-use-prediction-UTEC.git

# 2. Entrar a la carpeta del proyecto
cd Smarthphone-use-prediction-UTEC
```

🔴 Importante: Reconocimiento del repositorio

La URL del repositorio se guarda automáticamente en la carpeta oculta `.git`. No necesitas volver a pedir o escribir la URL de GitHub en los días posteriores.

2. Iniciar una Nueva Tarea (Creación de Rama)

Queda **estrictamente prohibido** programar o hacer cambios directamente sobre la rama `main`. Cada tarea o funcionalidad debe desarrollarse en su propia rama.

Bash

```
# 1. Asegurarte de estar en main y traer los últimos cambios del equipo
git checkout main
git pull origin main

# 2. Crear y cambiarte a tu propia rama de trabajo (ejemplo: feature-limpieza-datos)
git checkout -b feature-nombre-de-tu-tarea
```

💡 ¿Qué hacer mientras programas?

Una vez ejecutado el comando `git checkout -b`, puedes cerrar **Git Bash por completo**. Edita tus scripts en Positron, RStudio, VS Code o Jupyter durante las horas o días necesarios. Tus archivos quedan a salvo en tu disco duro.

3. Continuar el Trabajo al Día Siguiente o Tras Apagar la PC

Cuando retomes el trabajo al día siguiente o después de encender la computadora:

Bash

```
# 1. Navegar a la carpeta del repositorio
cd ruta/a/tu/repositorio/Smarthphone-use-prediction-UTEC

# 2. Confirmar en qué rama estás (debe mostrar feature-nombre-de-tu-tarea)
git status
```

¿Cómo sabe Git Bash en qué repositorio estás?

Git Bash no adivina la ubicación automáticamente. Al usar el comando `cd` para entrar a la carpeta del repositorio, la terminal detecta la configuración interna y te mostrará el nombre de tu rama entre paréntesis al lado del cursor.

4. Guardar y Subir tus Avances a GitHub

Las ramas no deben subirse por cambios diminutos o de 5 minutos; deben representar una tarea completada o un avance lógico significativo.

Bash

```
# 1. Verificar qué archivos han sido modificados o creados
git status

# 2. Preparar todos los cambios para el commit
git add .

# 3. Registrar el commit con un mensaje descriptivo
git commit -m "Implementar script de preprocesamiento de datos en R"

# 4. Subir tu rama a GitHub por primera vez
git push -u origin feature-nombre-de-tu-tarea
```

5. Revisión en GitHub y Fusión a main (Pull Request)

1. Ingresa al repositorio en GitHub: <https://github.com/frankrubio/Smarthphone-use-prediction-UTEC>.
2. Haz clic en el botón amarillo **"Compare & pull request"**.
3. Escribe una breve explicación de lo que realiza tu código y asigna a un compañero para revisión.
4. Una vez revisado y aprobado el código, presiona **"Merge pull request"** y confirma con **"Confirm merge"**.
5. Presiona el botón **"Delete branch"** en GitHub para borrar la rama remota ya unificada.

6. Limpieza Local y Preparación para el Siguiete Ciclo

Una vez que tu rama fue integrada en GitHub, debes sincronizar tu máquina local y eliminar la rama usada:

Bash

```
# 1. Regresar a la rama principal local
git checkout main

# 2. Descargar la versión actualizada con tu rama ya integrada
git pull origin main

# 3. Eliminar la rama local para mantener limpio tu entorno
git branch -d feature-nombre-de-tu-tarea
```

7. Manejo de Entornos Virtuales (Python y R)

Para evitar conflictos entre versiones de librerías globales y garantizar reproducibilidad entre Positron, RStudio y VS Code, el equipo utilizará entornos aislados. Puedes crearlo mediante tu IDE favorito o por consola.

A. Configuración de Python en IDE (VS Code / Positron / PyCharm)

La mayoría de IDEs permiten crear el entorno virtual de forma gráfica en pocos pasos:

- **VS Code / Positron:** Presiona `Ctrl + Shift + P` \$ ightarrow\$ selecciona **"Python: Create Environment"** \$ ightarrow\$ elige **Venv** \$ ightarrow\$ selecciona tu intérprete de Python \$ ightarrow\$ asegúrate de colocar como nombre de la carpeta `.venv` dentro de la raíz del proyecto.
- **PyCharm:** Ve a **Settings/Preferences** \$ ightarrow\$ **Project** \$ ightarrow\$ **Python Interpreter** \$ ightarrow\$ **Add Interpreter** \$ ightarrow\$ **Add Local Interpreter** \$ ightarrow\$ selecciona **Virtualenv Environment** (Location: `.venv`).

B. Alternativa por Consola para Python (Entorno `.venv`)

Bash

```
# 1. Crear entorno virtual local en la raíz del proyecto
python -m venv .venv

# 2. Activar en Windows (Git Bash)
source .venv/Scripts/activate

# 3. Instalar librerías y exportar dependencias
pip install pandas scikit-learn matplotlib
pip freeze > requirements.txt
```

C. Configuración para R (Librería renv)

R Console

```
# Inicializar entorno aislado en R dentro del proyecto
install.packages("renv")
renv::init()
```

🚫 REGLA DE ORO DE GIT: Archivos Ignorados

Independientemente de si creas el entorno con la interfaz gráfica del IDE o por la consola, las carpetas pesadas de ejecutables (`.venv/` , `renv/library/` , `__pycache__`) **NUNCA se suben a GitHub**. Deben figurar en el archivo `.gitignore` . Únicamente se suben y comparten los archivos ligeros de dependencias: `requirements.txt` (Python) y `renv.lock` (R).

8. Reglas y Convenciones del Proyecto

Categoría	Regla / Convención	Ejemplo Correcto	Ejemplo Incorrecto
Nombres de Archivos	Usar formato snake_case, minúsculas, sin espacios ni tildes.	<code>01_limpieza_datos.R</code> <code>modelo_python.py</code>	<code>script final (1).R</code> <code>LimpiezaDía1.py</code>
Mensajes de Commit	Redacción clara y descriptiva en imperativo. Puede usar mayúsculas.	<code>git commit -m "Agregar matriz de confusión"</code>	<code>git commit -m "cambios"</code> <code>git commit -m "asdasd"</code>
Rutas de Archivos	Usar únicamente rutas relativas al proyecto. Eliminar rutas absolutas.	<code>read.csv("data/raw.csv")</code>	<code>read.csv("C:/Users/MiPC/...")</code>
Seguridad	Prohibido subir credenciales, claves API o contraseñas en el código.	Variables de entorno / <code>.env</code>	<code>api_key = "123456"</code>